

Custom Instruction Design on Nios[®] V/g Processor

Agilex[™] 7 FPGA

Contents

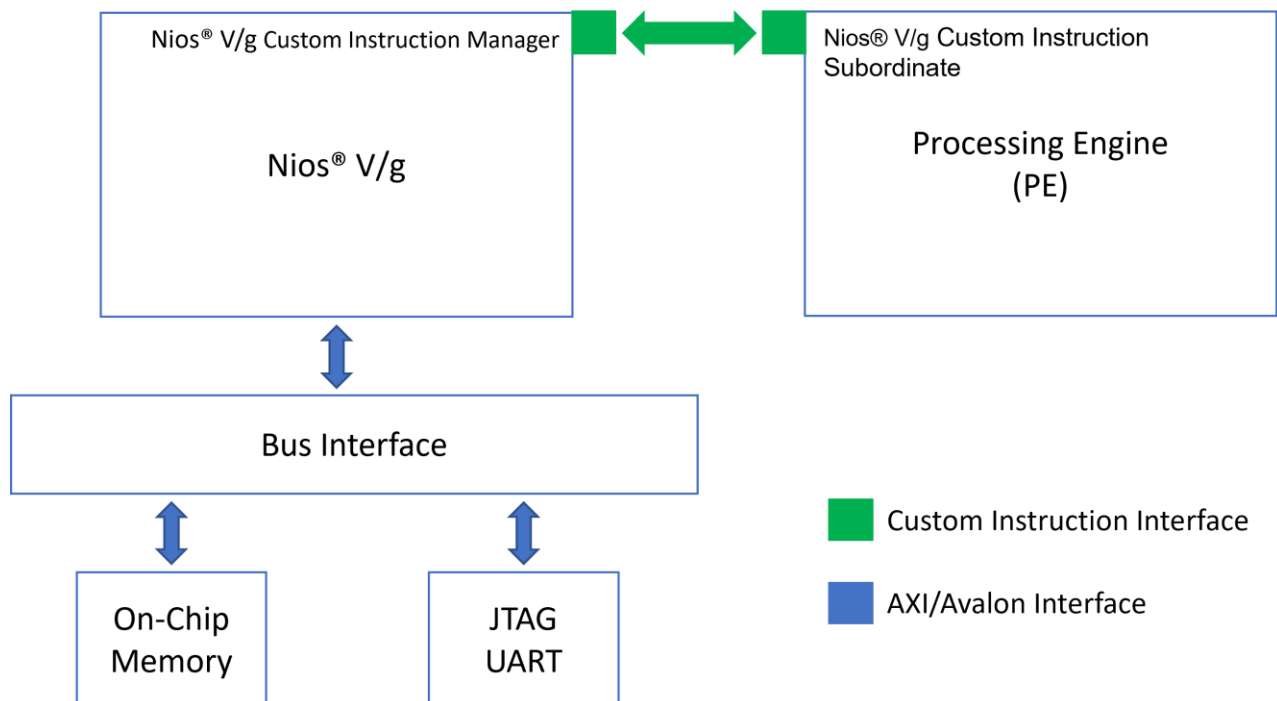
1. Theory of Operation.....	3
1.1. Block Diagram.....	3
1.2. Operations performed by Processing Engine (PE)	3
1.3. IP Cores	3
2. Directory Structure	4
3. Quick Start Guide	4
4. Generating the Design	5
4.1. Building Hardware Design	5
4.2. Building Software Design	5
5. Simulating the Design	6
6. Programming the Design	6
7. Expected Results	7

1. Theory of Operation

This design demonstrates the custom instruction feature of the Nios® V/g processor using Intel Agilex® 7 F-Series FPGA Development Kit.

1.1. Block Diagram

A Processing Engine (PE) that performs basic arithmetic and logical computations is connected to the Nios® V/g processor using the custom instruction interface. The current version of the Nios® V/g processor custom instruction interface supports operations up-to 32-Bit.



1.2. Operations performed by Processing Engine (PE)

A Processing Engine (PE) is connected to the Niosv/g processor using the custom instruction interface which performs the following 32-Bit arithmetic and logical operations:

- 1's complement
- 2's complement
- Multiplication
- Bit reversal (Reversing the order of Bits)
- Byte reversal (Reversing the order of Bytes)
- Word reversal (Reversing the order of words)
- Merge lower words (Combine/Merge the lower words of two inputs)
- Merge higher words (Combine/Merge the higher words of two inputs)

1.3. IP Cores

The following IPs are used in this design.

- Nios V/g soft processor core
- On Chip RAM-II
- JTAG UART
- Custom PE 1
- Custom PE 2
- System ID

2. Directory Structure

The directory structure is explained below:

Root directory: agf014ea-dev-devkit/niosv_g/ci_basic_operations

Directories	Content
docs	Document capturing the details of the example design
img	Block diagram for the example designs
ready_to_test	Prebuilt binaries (FPGA .sof file and Application .elf file) which can be programmed on the board and tested
sources	Files needed to create the design
- hw - scripts - sw/app	- Necessary hardware files (.sv, .v, .ip) of the design - This folder consists of scripts to build the design - This folder contains software application files

3. Quick Start Guide

You may try the example on the target development kit using the prebuilt binaries. The respective FPGA sof and Application elf files are found in ready_to_test folder.

Refer to [Chapter 6 Programming the Design](#) for the steps to program these files.

4. Generating the Design

The implementation of Nios® V processor system requires a hardware design developed using the Quartus® Prime software. After that, it is required to develop the software design that complements the processor system.

4.1. Building Hardware Design

Begin designing the system hardware by instantiating the Nios® V processor and its peripherals into Platform Designer. After configuring the system assignments and constraints, complete the hardware design by performing a successful compilation.

We will be using `sources/scripts/build_sof.py` to develop the hardware design.

This script will create a new platform design, add the IPs into the design, makes the connections, compiles the design and generates the `.sof` file.

Steps to execute the script are as follows:

1. Invoke the Nios V Command Shell in the terminal
2. Run the following command in the terminal from top level project directory:
`> quartus_py ./scripts/build_sof.py`
3. The Quartus tool will compile the design and generate the output files into `<Root Directory>/sources/hw/output_files`

4.2. Building Software Design

After the processor system is ready, you may begin building the software design. It consists of the following steps:

1. Create a board support package (BSP) project.
2. Create a Nios® V processor application project with source code.
3. Build the application.
4. Convert the application `.elf` file into `.hex` file for memory initialization.

To create software application, run the following commands in the terminal:

```
> niosv-bsp -c --quartus-project=hw/niosv_custom_instruction.qpf --  
qsys=hw/sys.qsys --type=hal sw/bsp/settings.bsp  
  
> niosv-app --bsp-dir=sw/bsp --app-dir=sw/app -- srcs=sw/app/custom_instr_app.c  
  
> niosv-shell  
  
> cmake -S ./sw/app -G "Unix Makefiles" -B sw/app/build  
  
> make -C sw/app/build
```

Note: It is recommended to clean the `app/build` project before regenerating `elf` (`cmake` and `make`).

5. Simulating the Design

You may carry on simulation for this design to evaluate its intended functions. During simulation, the action of downloading .elf file is unable to be simulated. Thus, the processor memory is instead initialized with the application .hex file.

Use the following commands to run the simulation:

```
# Generate Testbench from Platform Designer.

# Generate -> Generate Testbench System

> cp ./hw/onchip_mem.hex ./sys_tb/sys_tb/sim/mentor

> cd hw/sys_tb/sys_tb/sim/mentor/

> vsim &

> source msim_setup.tcl

> ld_debug

> run -all
```

6. Programming the Design

Use the Quartus® Prime Programmer tool and the Ashling* RiscFree* IDE to program the Nios® V processor-based system into the FPGA and to run your application.

Program the generated sof, then download the elf file on the board according to the command below.

```
> quartus_pgm --cable=1 -m jtag -o 'p;ready_to_test/niosv_custom_instruction.sof'

#----- Download the elf file on the board ----#

> niosv-download -g ready_to_test/app.elf -c 1

#----- Verify the output on the terminal by using the following command in the
terminal -----#

> juart-terminal -c 1 -i 0
```

Note: Reduce the JTAG clock frequency to 6MHz before programming the application .elf file on the board using the command: `jtagconfig --setparam 1 JtagClock 6M`

7. Expected Results

The following is the output as observed on the JTAG UART terminal. The output is analogous to the logic from the application code. Users should be able to observe the same output on their terminal/setup.

Figure 1-2. Expected Result

```
Now running custom instruction basic test...
Hello world
*****

PE1 operations
*****

1's complement of DATA1:0xff
Expected Output:ffffff00
Actual Output:ffffff00
*****

2's complement of DATA1:0xff
Expected Output:ffffff01
Actual Output:ffffff01
*****

Multiplication of DATA1:0x1f and DATA2:0x80
Expected Output:f80
Actual Output:f80
*****

Bit Reversal of DATA1:0x1f
Expected Output:f8000000
Actual Output:f8000000
*****

Byte Reversal of DATA1:0x1f
Expected Output:1f000000
Actual Output:1f000000
*****

Word Reversal of DATA1:0x1f
Expected Output:1f0000
Actual Output:1f0000
*****

Merge Lower-dword DATA1:0x74009078 and DATA2:0x82007083
Expected Output:90787083
Actual Output:90787083
*****

Merge Higher-dword of DATA1:0x74009078 and DATA2:0x82007083
Expected Output:74008200
Actual Output:74008200
End of PE1 operations
*****
```



```

*****
PE2 operations
*****
1's complement of DATA1:0xff
Expected Output:ffffff00
Actual Output:ffffff00
*****
2's complement of DATA1:0xff
Expected Output:ffffff01
Actual Output:ffffff01
*****
Multiplication of DATA1:0x1f and DATA2:0x80
Expected Output:f80
Actual Output:f80
*****
Bit Reversal of DATA1:0x111fff
Expected Output:fff88800
Actual Output:fff88800
*****
Byte Reversal of DATA1:0x111fff
Expected Output:ff1f1100
Actual Output:ff1f1100
*****
Word Reversal of DATA1:0x111fff
Expected Output:1fff0011
Actual Output:1fff0011
*****
Merge Lower-dword of DATA1:0x74009078 and DATA2:0x82007083
Expected Output:90787083
Actual Output:90787083
*****
Merge Higher-dword of DATA1:0x74009078 and DATA2:0x82007083
Expected Output:74008200
Actual Output:74008200
End of PE2 operations
*****
Bye world!

```